

Task 9: Main project - Using ansible

DESCRIPTION

Instructions:

1. Install the necessary packages, which include MySQL 10.6, PHP version 8 or higher, and Nginx on your Linux server.
 2. Create a user's home directory under the '/home' directory. You should set the root directory for the website to be '/home/username/websitename/public.'
 3. Configure SFTP (Secure File Transfer Protocol) access for the user to ensure secure file management.
 4. Configure phpmyadmin access for the user to ensure secure database management.
 4. Install a free SSL certificate to enable secure HTTPS access to your website.
 5. Once the website is hosted, proceed to the WordPress dashboard to make necessary configurations and customizations.
 6. Write a personal blog post about yourself using the WordPress content management system.
-

OBJECTIVE

The objective of this project was to automate the deployment of a WordPress website using Ansible on an AWS EC2 instance. The project involved:

- Installing Nginx, PHP, and MariaDB.
- Creating a dedicated website user.
- Configuring SFTP access.
- Creating and configuring a WordPress database.
- Installing WordPress.
- Configuring phpMyAdmin.
- Creating a personal blog post.
- Configuring a DuckDNS subdomain.
- Securing the website using a free SSL certificate from Let's Encrypt.

ARCHITECTURE OVERVIEW

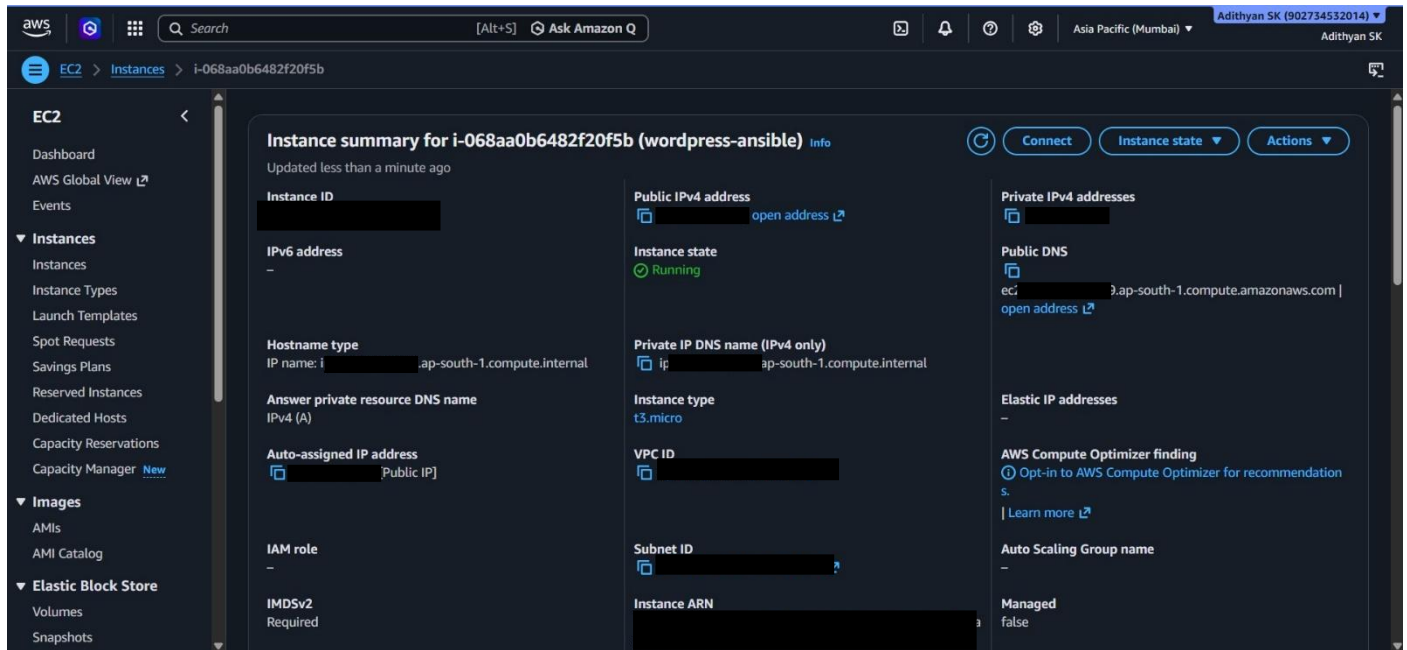
The solution was implemented on an AWS EC2 instance running Amazon Linux 2023. Ansible was used to automate the deployment and configuration of the web stack, database services, WordPress installation, user management, phpMyAdmin access, and SSL configuration.

Components Used:

1. AWS EC2
2. Ansible
3. Nginx
4. PHP 8.5
5. MariaDB 10.5
6. WordPress
7. phpMyAdmin
8. DuckDNS
9. Let's Encrypt SSL

Task 1: AWS Environment Setup

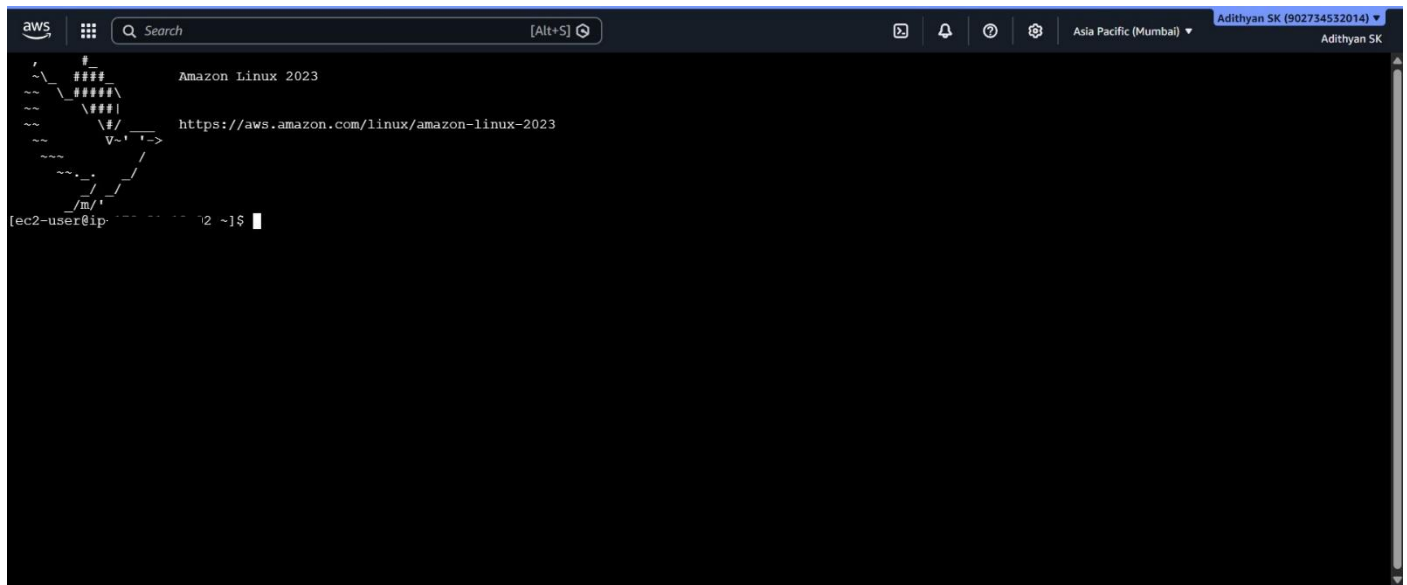
Step 1: Launch AWS EC2 Instance



The screenshot displays the AWS Management Console interface for an EC2 instance. The left sidebar shows the navigation menu with categories like EC2, Images, and Elastic Block Store. The main content area is titled "Instance summary for i-068aa0b6482f20f5b (wordpress-ansible)". It provides a comprehensive overview of the instance's configuration and status.

Property	Value
Instance ID	i-068aa0b6482f20f5b
Public IPv4 address	54.156.141.123
Private IPv4 addresses	10.0.1.15
Instance state	Running
Public DNS	ec2-54-156-141-123.ap-south-1.compute.amazonaws.com
Private IP DNS name (IPv4 only)	ip-10-0-1-15.ap-south-1.compute.internal
Instance type	t3.micro
VPC ID	vpc-0a519950
Subnet ID	subnet-0a519950
Instance ARN	arn:aws:ec2:ap-south-1:902734532014:instance/i-068aa0b6482f20f5b
IPV6 address	-
Hostname type	IP name: ip-10-0-1-15.ap-south-1.compute.internal
Answer private resource DNS name	IPV4 (A)
Auto-assigned IP address	10.0.1.15 [Public IP]
IAM role	-
IMDSv2	Required
Elastic IP addresses	-
AWS Compute Optimizer finding	Opt-in to AWS Compute Optimizer for recommendation s. Learn more
Auto Scaling Group name	-
Managed	false

Step 2: Connect to the EC2 Instance



The screenshot shows a terminal window with the AWS CLI interface. The prompt is [ec2-user@ip-10-0-1-15 ~]\$ and the command being executed is curl https://aws.amazon.com/linux/amazon-linux-2023. The output of the command is Amazon Linux 2023, indicating a successful connection to the instance.

```
[ec2-user@ip-10-0-1-15 ~]$ curl https://aws.amazon.com/linux/amazon-linux-2023
Amazon Linux 2023
```

Step 3: Install Ansible

```
[ec2-user@ip-172-31-12-92 ~]$ sudo dnf install ansible -y
Amazon Linux 2023 Kernel Livepatch repository                               390 kB/s | 47 kB    00:00
Dependencies resolved.

Package                               Architecture      Version            Repository          Size
-----
Installing:
ansible                               noarch            8.3.0-1.amzn2023.0.2  amazonlinux        32 M
Installing dependencies:
ansible-core                          x86_64            2.15.3-1.amzn2023.0.11  amazonlinux        2.5 M
git-core                             x86_64            2.50.1-1.amzn2023.0.1  amazonlinux        4.9 M
sshpas                               x86_64            1.09-6.amzn2023.0.1    amazonlinux        28 k

Transaction Summary
-----
Install 4 Packages

Total download size: 40 M
Installed size: 552 M
Downloading Packages:
(1/4): ansible-core-2.15.3-1.amzn2023.0.11.x86_64.rpm                    37 MB/s | 2.5 MB    00:00
(2/4): git-core-2.50.1-1.amzn2023.0.1.x86_64.rpm                       46 MB/s | 4.9 MB    00:00
(3/4): sshpass-1.09-6.amzn2023.0.1.x86_64.rpm                          652 kB/s | 28 kB    00:00
(4/4): ansible-8.3.0-1.amzn2023.0.2.noarch.rpm                         67 MB/s | 32 MB    00:00
Total                                                                    76 MB/s | 40 MB    00:00

Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing      :
  Installing     : sshpass-1.09-6.amzn2023.0.1.x86_64                    1/1
                                                         1/4
```

Step 4: Verify Ansible Installation

```
[ec2-user@ip-172-31-12-92 ~]$ ansible --version
ansible [core 2.15.3]
  config file = None
  configured module search path = ['/home/ec2-user/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3.9/site-packages/ansible
  ansible collection location = /home/ec2-user/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.9.25 (main, May 25 2026, 00:00:00) [GCC 11.5.0 20240719 (Red Hat 11.5.0-5)] (/usr/bin/python3.9)
  jinja version = 3.1.4
  libyaml = True
[ec2-user@ip-172-31-12-92 ~]$
```

Task 2: Ansible Project Initialization

Step 1: Create Project Directory

```
[ec2-user@ip-172-31-12-92 ~]$ mkdir ~/wordpress-project
[ec2-user@ip-172-31-12-92 ~]$ cd ~/wordpress-project
[ec2-user@ip-172-31-12-92 wordpress-project]$ pwd
/home/ec2-user/wordpress-project
[ec2-user@ip-172-31-12-92 wordpress-project]$
```

Step 2: Create Ansible Inventory File

```
[ec2-user@ip-172-31-12-92 wordpress-project]$ vim inventory
[ec2-user@ip-172-31-12-92 wordpress-project]$ cat inventory
[web]
localhost ansible connection=local ansible python interpreter=/usr/bin/python3
```

Step 3: Verify Inventory Configuration

```
[ec2-user@ip-172-31-12-92 wordpress-project]$ ansible -i inventory web -m ping
[WARNING]: Platform linux on host localhost is using the discovered Python interpreter at /usr/bin/python3.9, but future installation of another Python interpreter could change the meaning of that path. See https://docs.ansible.com/ansible-core/2.15/reference_appendices/interpreter_discovery.html for more information.
localhost | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3.9"
  },
  "changed": false,
  "ping": "pong"
}
```

The inventory file was configured to manage the local EC2 instance using Ansible.

Task 3: Web Stack Deployment using Ansible

Step 1: Create Initial Web Stack Playbook

```
[ec2-user@ip-172-31-12-92 wordpress-project]$ vim webstack.yml
[ec2-user@ip-172-31-12-92 wordpress-project]$ cat webstack.yml
---
- name: Install Nginx
  hosts: web
  become: yes

  tasks:
    - name: Install Nginx
      dnf:
        name: nginx
        state: present

    - name: Start Nginx
      service:
        name: nginx
        state: started
        enabled: yes
```

Step 2: Execute Web Stack Playbook

```
[ec2-user@ip-172-31-12-92 wordpress-project]$ ansible-playbook -i inventory webstack.yml

PLAY [Install Nginx] *****

TASK [Gathering Facts] *****
[WARNING]: Platform linux on host localhost is using the discovered Python interpreter at /usr/bin/python3.9, but future installation of another Python interpreter
could change the meaning of that path. See https://docs.ansible.com/ansible-core/2.15/reference_appendices/interpreter_discovery.html for more information.
ok: [localhost]

TASK [Install Nginx] *****
changed: [localhost]

TASK [Start Nginx] *****
changed: [localhost]

PLAY RECAP *****
localhost                : ok=3    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Step 3: Verify Nginx Installation

```
[ec2-user@ip-172-31-12-92 wordpress-project]$ sudo systemctl status nginx
● nginx.service - The nginx HTTP and reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: disabled)
   Active: active (running) since Wed 2026-06-10 06:12:04 UTC; 33s ago
     Process: 26975 ExecStartPre=/usr/bin/rm -f /run/nginx.pid (code=exited, status=0/SUCCESS)
     Process: 26976 ExecStartPre=/usr/sbin/nginx -t (code=exited, status=0/SUCCESS)
     Process: 26977 ExecStart=/usr/sbin/nginx (code=exited, status=0/SUCCESS)
   Main PID: 26978 (nginx)
    Tasks: 3 (limit: 1014)
   Memory: 3.4M
      CPU: 61ms
   CGroup: /system.slice/nginx.service
           └─26978 "nginx: master process /usr/sbin/nginx"
             └─26979 "nginx: worker process"
               └─26980 "nginx: worker process"

Jun 10 06:12:04 ip-172-31-12-92.ap-south-1.compute.internal systemd[1]: Starting nginx.service - The nginx HTTP and reverse proxy server...
Jun 10 06:12:04 ip-172-31-12-92.ap-south-1.compute.internal nginx[26976]: nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
Jun 10 06:12:04 ip-172-31-12-92.ap-south-1.compute.internal nginx[26976]: nginx: configuration file /etc/nginx/nginx.conf test is successful
Jun 10 06:12:04 ip-172-31-12-92.ap-south-1.compute.internal systemd[1]: Started nginx.service - The nginx HTTP and reverse proxy server.
```

Step 4: Extend Playbook for Complete LEMP Stack

```
--
- name: Configure Web Server
  hosts: web
  become: yes

  tasks:

    - name: Install packages
      dnf:
        name:
          - nginx
          - php
          - php-fpm
          - php-mysqlnd
          - mariadb105-server
        state: present

    - name: Start nginx
      service:
        name: nginx
        state: started
        enabled: yes

    - name: Start MariaDB
      service:
        name: mariadb
        state: started
        enabled: yes

    - name: Start PHP-FPM
      service:
        name: php-fpm

        state: started
        enabled: yes

    - name: Create website user
      user:
        name: websiteuser
        shell: /bin/bash

    - name: Create website directory
      file:
        path: /home/websiteuser/myblog/public
        state: directory
        owner: websiteuser
        group: websiteuser
        mode: '0755'
```

Playbook includes

- *Nginx*
- *PHP 8*
- *MariaDB*
- *phpMyAdmin*
- *Website user creation*

Step 5: Execute Updated Playbook

```
[ec2-user@ip-172-31-12-92 wordpress-project]$ ansible-playbook -i inventory webstack.yml

PLAY [Configure Web Server] *****

TASK [Gathering Facts] *****
[WARNING]: Platform linux on host localhost is using the discovered Python interpreter at /usr/bin/python3.9, but future installation of another Python interpreter
could change the meaning of that path. See https://docs.ansible.com/ansible-core/2.15/reference_appendices/interpreter_discovery.html for more information.
ok: [localhost]

TASK [Install packages] *****
changed: [localhost]

TASK [Start nginx] *****
ok: [localhost]

TASK [Start MariaDB] *****
changed: [localhost]

TASK [Start PHP-FPM] *****
changed: [localhost]

TASK [Create website user] *****
changed: [localhost]

TASK [Create website directory] *****
changed: [localhost]

PLAY RECAP *****
localhost : ok=7  changed=5  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
```

Step 6: Verify PHP and MariaDB Services

```
[ec2-user@ip-172-31-12-92 wordpress-project]$ php -v
PHP 8.5.6 (cli) (built: May 5 2026 21:19:36) (NTS gcc x86_64)
Copyright (c) The PHP Group
Built by Amazon Linux
Zend Engine v4.5.6, Copyright (c) Zend Technologies
    with Zend OPcache v8.5.6, Copyright (c), by Zend Technologies
[ec2-user@ip-172-31-12-92 wordpress-project]$ sudo systemctl status mariadb
● mariadb.service - MariaDB 10.5 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; preset: disabled)
   Active: active (running) since Wed 2026-06-10 06:20:36 UTC; 1min 8s ago
     Docs: man:mariadb(8)
           https://mariadb.com/kb/en/library/systemd/
   Main PID: 29462 (mariabdd)
    Status: "Taking your SQL requests now..."
     Tasks: 8 (limit: 1014)
    Memory: 66.6M
       CPU: 560ms
    CGroup: /system.slice/mariadb.service
            └─29462 /usr/libexec/mariabdd --basedir=/usr

Jun 10 06:20:36 ip-172-31-12-92.ap-south-1.compute.internal mariadb-prepare-db-dir[29417]: The second is mysql@localhost, it has no password either, but
Jun 10 06:20:36 ip-172-31-12-92.ap-south-1.compute.internal mariadb-prepare-db-dir[29417]: you need to be the system 'mysql' user to connect.
Jun 10 06:20:36 ip-172-31-12-92.ap-south-1.compute.internal mariadb-prepare-db-dir[29417]: After connecting you can set the password, if you would need to be
Jun 10 06:20:36 ip-172-31-12-92.ap-south-1.compute.internal mariadb-prepare-db-dir[29417]: able to connect as any of these users with a password and without sudo
Jun 10 06:20:36 ip-172-31-12-92.ap-south-1.compute.internal mariadb-prepare-db-dir[29417]: See the MariaDB Knowledgebase at https://mariadb.com/kb
Jun 10 06:20:36 ip-172-31-12-92.ap-south-1.compute.internal mariadb-prepare-db-dir[29417]: Please report any problems at https://mariadb.org/jira
Jun 10 06:20:36 ip-172-31-12-92.ap-south-1.compute.internal mariadb-prepare-db-dir[29417]: The latest information about MariaDB is available at https://mariadb.org/.
Jun 10 06:20:36 ip-172-31-12-92.ap-south-1.compute.internal mariadb-prepare-db-dir[29417]: Consider joining MariaDB's strong and vibrant community:
Jun 10 06:20:36 ip-172-31-12-92.ap-south-1.compute.internal mariadb-prepare-db-dir[29417]: https://mariadb.org/get-involved/
Jun 10 06:20:36 ip-172-31-12-92.ap-south-1.compute.internal systemd[1]: Started mariadb.service - MariaDB 10.5 database server.
```

Step 7: Verify Website User Ownership

```
[ec2-user@ip-172-31-12-92 wordpress-project]$ ls -ld /home/websiteuser
drwx-----. 3 websiteuser websiteuser 76 Jun 10 06:20 /home/websiteuser
[ec2-user@ip-172-31-12-92 wordpress-project]$ ls -ld /home/websiteuser/myblog/public
ls: cannot access '/home/websiteuser/myblog/public': Permission denied
[ec2-user@ip-172-31-12-92 wordpress-project]$
```

The website directory is owned by the website user and access restrictions are functioning correctly, preventing unauthorized users from viewing the contents.

Task 4: Database Configuration

Step 1: Install community.mysql Collection

```
[ec2-user@ip-172-31-12-92 wordpress-project]$ ansible-galaxy collection install community.mysql
Starting galaxy collection install process
Nothing to do. All requested collections are already installed. If you want to reinstall them, consider using '--force'.
[ec2-user@ip-172-31-12-92 wordpress-project]$ ansible-galaxy collection list | grep mysql
community.mysql          3.7.2
```

Step 2: Create Database Automation Playbook

```
[ec2-user@ip-172-31-12-92 wordpress-project]$ vim database.yml
[ec2-user@ip-172-31-12-92 wordpress-project]$ cat database.yml
---
- name: Configure WordPress Database
  hosts: web
  become: yes

  tasks:
    - name: Create database
      shell: |
        mysql -e "CREATE DATABASE IF NOT EXISTS wordpresdb;"

    - name: Create user
      shell: |
        mysql -e "CREATE USER IF NOT EXISTS 'wpuser'@'localhost' IDENTIFIED BY 'StrongPassword123!';"

    - name: Grant privileges
      shell: |
        mysql -e "GRANT ALL PRIVILEGES ON wordpresdb.* TO 'wpuser'@'localhost'; FLUSH PRIVILEGES;"
```

Step 3: Execute Database Playbook

```
[ec2-user@ip-172-31-12-92 wordpress-project]$ ansible-playbook -i inventory database.yml

PLAY [Configure WordPress Database] *****

TASK [Gathering Facts] *****
ok: [localhost]

TASK [Create database] *****
changed: [localhost]

TASK [Create user] *****
changed: [localhost]

TASK [Grant privileges] *****
changed: [localhost]

PLAY RECAP *****
localhost                : ok=4    changed=3    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Step 4: Verify Database and User Creation

```
[ec2-user@ip-172-31-12-92 wordpress-project]$ sudo mysql
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 7
Server version: 10.5.29-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| wordpressdb |
+-----+
4 rows in set (0.002 sec)

MariaDB [(none)]> SELECT User FROM mysql.user;
+-----+
| User |
+-----+
| mariadb.sys |
| mysql |
| root |
| wpuser |
+-----+
4 rows in set (0.008 sec)

MariaDB [(none)]> exit;
Bye
```

The playbook automatically created the WordPress database, database user, and assigned the required privileges.

Task 5: WordPress Deployment

Step 1: Create WordPress Deployment Playbook

```
[ec2-user@ip-172-31-12-92 wordpress-project]$ vim wordpress.yml
[ec2-user@ip-172-31-12-92 wordpress-project]$ cat wordpress.yml
---
- name: Install WordPress
  hosts: web
  become: yes

  tasks:

    - name: Download WordPress
      get_url:
        url: https://wordpress.org/latest.tar.gz
        dest: /tmp/wordpress.tar.gz

    - name: Extract WordPress
      unarchive:
        src: /tmp/wordpress.tar.gz
        dest: /tmp
        remote_src: yes

    - name: Copy WordPress files
      copy:
        src: /tmp/wordpress/
        dest: /home/websiteuser/myblog/public/
        owner: websiteuser
        group: websiteuser
        mode: '0755'
        remote_src: yes
```

Step 2: Execute WordPress Playbook

```
[ec2-user@ip-172-31-12-92 wordpress-project]$ ansible-playbook -i inventory wordpress.yml

PLAY [Install WordPress] *****
TASK [Gathering Facts] *****
ok: [localhost]

TASK [Download WordPress] *****
changed: [localhost]

TASK [Extract WordPress] *****
changed: [localhost]

TASK [Copy WordPress files] *****
changed: [localhost]

PLAY RECAP *****
localhost                : ok=4    changed=3    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Step 3: Verify WordPress Files

```
[ec2-user@ip-172-31-12-92 wordpress-project]$ sudo ls /home/websiteuser/myblog/public
index.php  readme.html  wp-admin      wp-comments-post.php  wp-content  wp-includes  wp-load.php  wp-mail.php  wp-signup.php  xmlrpc.php
license.txt wp-activate.php wp-blog-header.php wp-config-sample.php wp-cron.php wp-links-opml.php wp-login.php wp-settings.php wp-trackback.php
```

Step 4: Configure Nginx Virtual Host

```
[ec2-user@ip-172-31-12-92 phpmyadmin]$ sudo vim /etc/nginx/conf.d/wordpress.conf
[ec2-user@ip-172-31-12-92 phpmyadmin]$ sudo cat /etc/nginx/conf.d/wordpress.conf
server {
    listen 80;
    server_name 13.203.66.209;

    root /home/websiteuser/myblog/public;
    index index.php index.html;

    location /phpmyadmin {
        alias /usr/share/nginx/html/phpmyadmin;
        index index.php;
    }

    location ~ ^/phpmyadmin/(.+\.php)$ {
        alias /usr/share/nginx/html/phpmyadmin/$1;
        include fastcgi_params;
        fastcgi_pass unix:/run/php-fpm/www.sock;
        fastcgi_param SCRIPT_FILENAME /usr/share/nginx/html/phpmyadmin/$1;
    }

    location / {
        try_files $uri $uri/ /index.php?$args;
    }

    location ~ \.php$ {
        include fastcgi_params;
        fastcgi_pass unix:/run/php-fpm/www.sock;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
        include fastcgi.conf;
    }

}

location ~ /\.ht {
    deny all;
}
}}
```

```
[ec2-user@ip-172-31-12-92 phpmyadmin]$
```

Step 5: Validate Nginx Configuration

```
[ec2-user@ip-172-31-12-92 wordpress-project]$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

Step 6: Restart Nginx and PHP-FPM Services

```
[ec2-user@ip-172-31-12-92 wordpress-project]$ sudo systemctl restart nginx
[ec2-user@ip-172-31-12-92 wordpress-project]$ sudo systemctl restart php-fpm
[ec2-user@ip-172-31-12-92 wordpress-project]$ sudo systemctl status nginx
● nginx.service - The nginx HTTP and reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: disabled)
   Drop-In: /usr/lib/systemd/system/nginx.service.d
            └─php-fpm.conf
   Active: active (running) since Wed 2026-06-10 07:08:12 UTC; 23s ago
     Process: 33479 ExecStartPre=/usr/bin/rm -f /run/nginx.pid (code=exited, status=0/SUCCESS)
     Process: 33480 ExecStartPre=/usr/sbin/nginx -t (code=exited, status=0/SUCCESS)
     Process: 33481 ExecStart=/usr/sbin/nginx (code=exited, status=0/SUCCESS)
   Main PID: 33482 (nginx)
    Tasks: 3 (limit: 1014)
   Memory: 3.5M
     CPU: 61ms
   CGroup: /system.slice/nginx.service
           └─33482 "nginx: master process /usr/sbin/nginx"
             └─33483 "nginx: worker process"
               └─33484 "nginx: worker process"

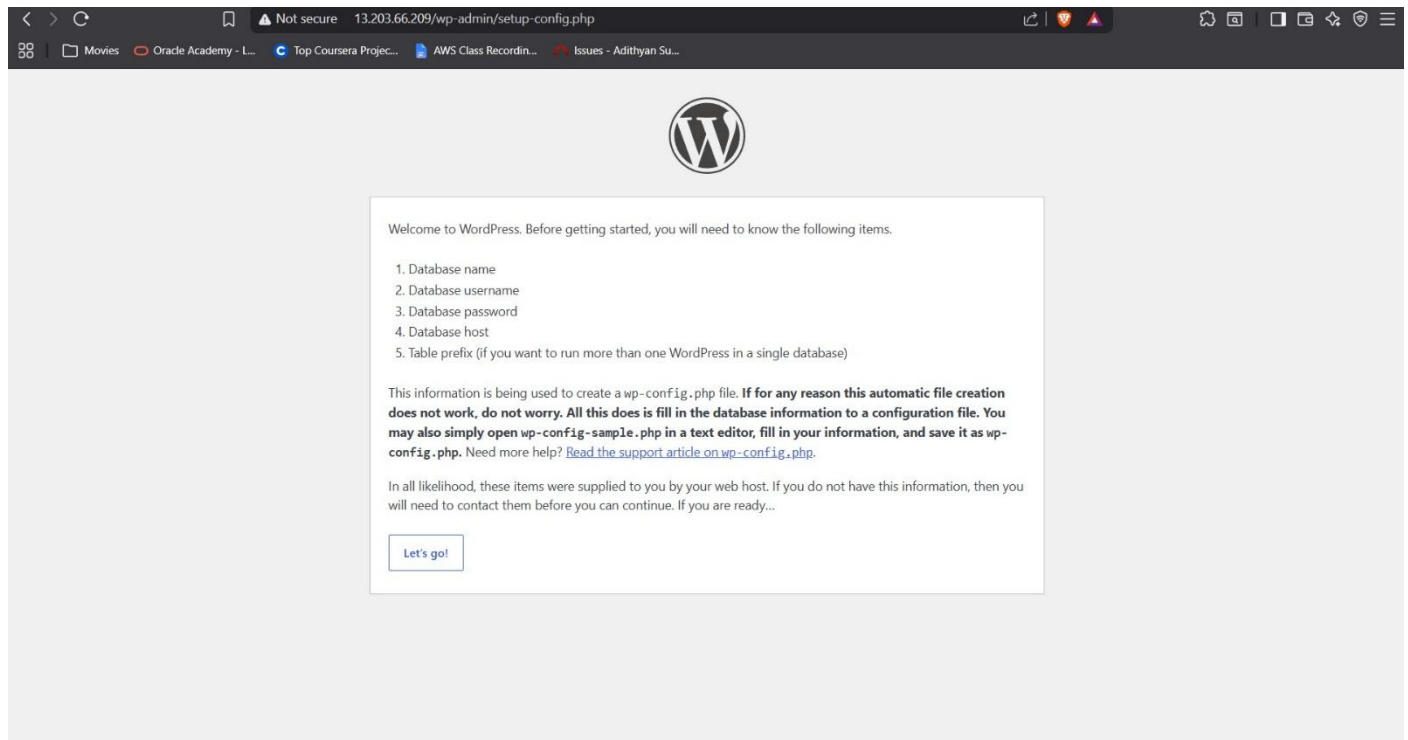
Jun 10 07:08:12 ip-172-31-12-92.ap-south-1.compute.internal systemd[1]: Starting nginx.service - The nginx HTTP and reverse proxy server...
Jun 10 07:08:12 ip-172-31-12-92.ap-south-1.compute.internal nginx[33480]: nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
Jun 10 07:08:12 ip-172-31-12-92.ap-south-1.compute.internal nginx[33480]: nginx: configuration file /etc/nginx/nginx.conf test is successful
Jun 10 07:08:12 ip-172-31-12-92.ap-south-1.compute.internal systemd[1]: Started nginx.service - The nginx HTTP and reverse proxy server.
[ec2-user@ip-172-31-12-92 wordpress-project]$ sudo systemctl status php-fpm
● php-fpm.service - The PHP FastCGI Process Manager
   Loaded: loaded (/usr/lib/systemd/system/php-fpm.service; enabled; preset: disabled)
   Active: active (running) since Wed 2026-06-10 07:08:18 UTC; 24s ago
     Main PID: 33492 (php-fpm)
   Status: "Processes active: 0, idle: 5, Requests: 0, slow: 0, Traffic: 0.00req/sec"
```

Step 7: Configure Website Permissions

```
[ec2-user@ip-172-31-12-92 wordpress-project]$ sudo chmod 755 /home/websiteuser
[ec2-user@ip-172-31-12-92 wordpress-project]$ namei -l /home/websiteuser/myblog/public
f: /home/websiteuser/myblog/public
dr-xr-xr-x root      root      /
drwxr-xr-x root      root      home
drwxr-xr-x websiteuser websiteuser websiteuser
drwxr-xr-x websiteuser websiteuser myblog
drwxr-xr-x websiteuser websiteuser public
```

Task 6: WordPress Setup and Customization

Step 1: Access WordPress Installation Wizard



Step 2: Configure WordPress Installation

Unable to write to wp-config.php file.

You can create the wp-config.php file manually and paste the following text into it.

Configuration rules for wp-config.php:

```
<?php
/**
 * The base configuration for WordPress
 *
 * The wp-config.php creation script uses this file during the installation.
 * You don't have to use the website, you can copy this file to "wp-config.php"
 * and fill in the values.
 *
 * This file contains the following configurations:
 *
 * * Database settings
 * * Secret keys
 * * Database table prefix
 * * ABSPATH
 *
 * @link https://codex.wordpress.org/Editing_wp-config.php
 */
```

After you've done that, click "Run the installation".

[Run the installation](#)

Welcome

Welcome to the famous five-minute WordPress installation process! Just fill in the information below and you'll be on your way to using the most extendable and powerful personal publishing platform in the world.

Information needed

Please provide the following information. Do not worry, you can always change these settings later.

Site Title

Username

Names can have only alphanumeric characters, spaces, underscores, hyphens, periods, and the @ symbol.

Password [Hide](#)

Important: You will need this password to log in. Please store it in a secure location.

Your Email

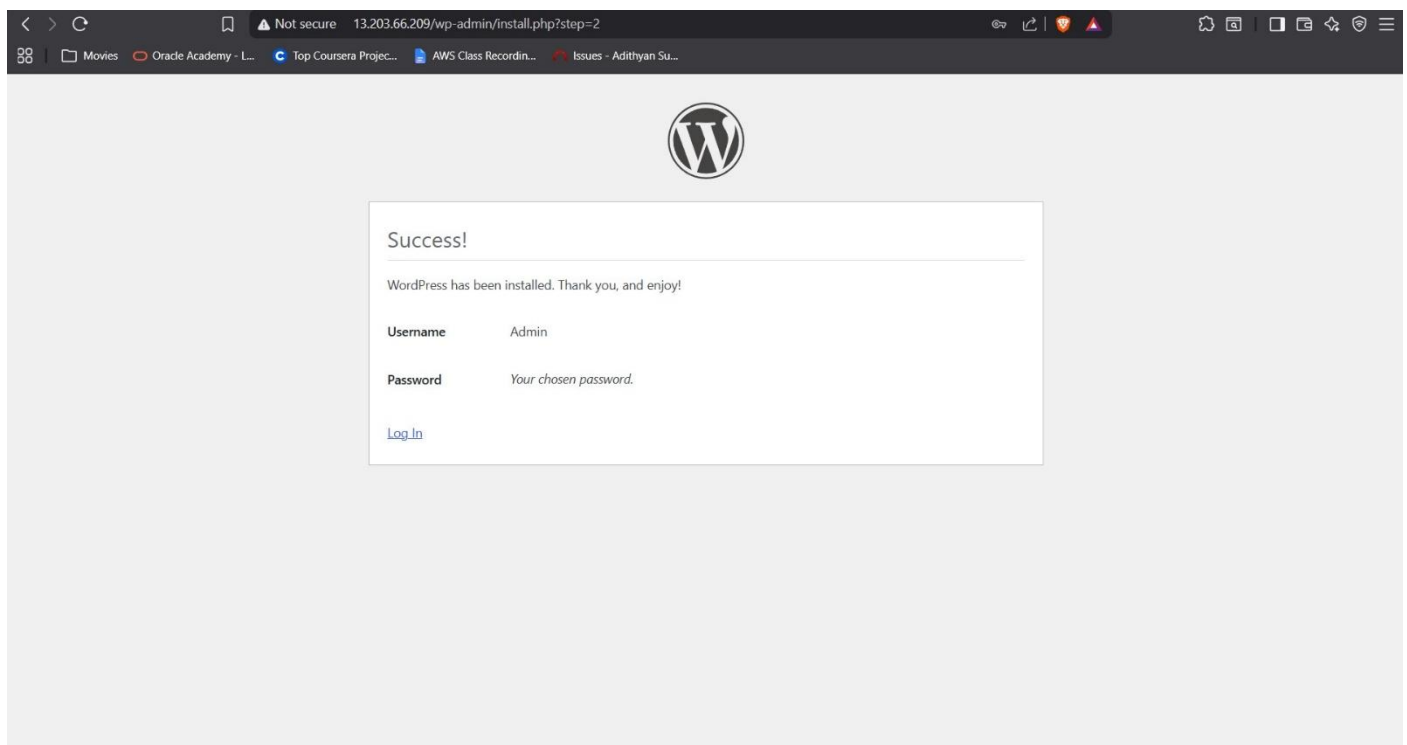
Double-check your email address before continuing.

Search engine visibility ☐ Discourage search engines from indexing this site

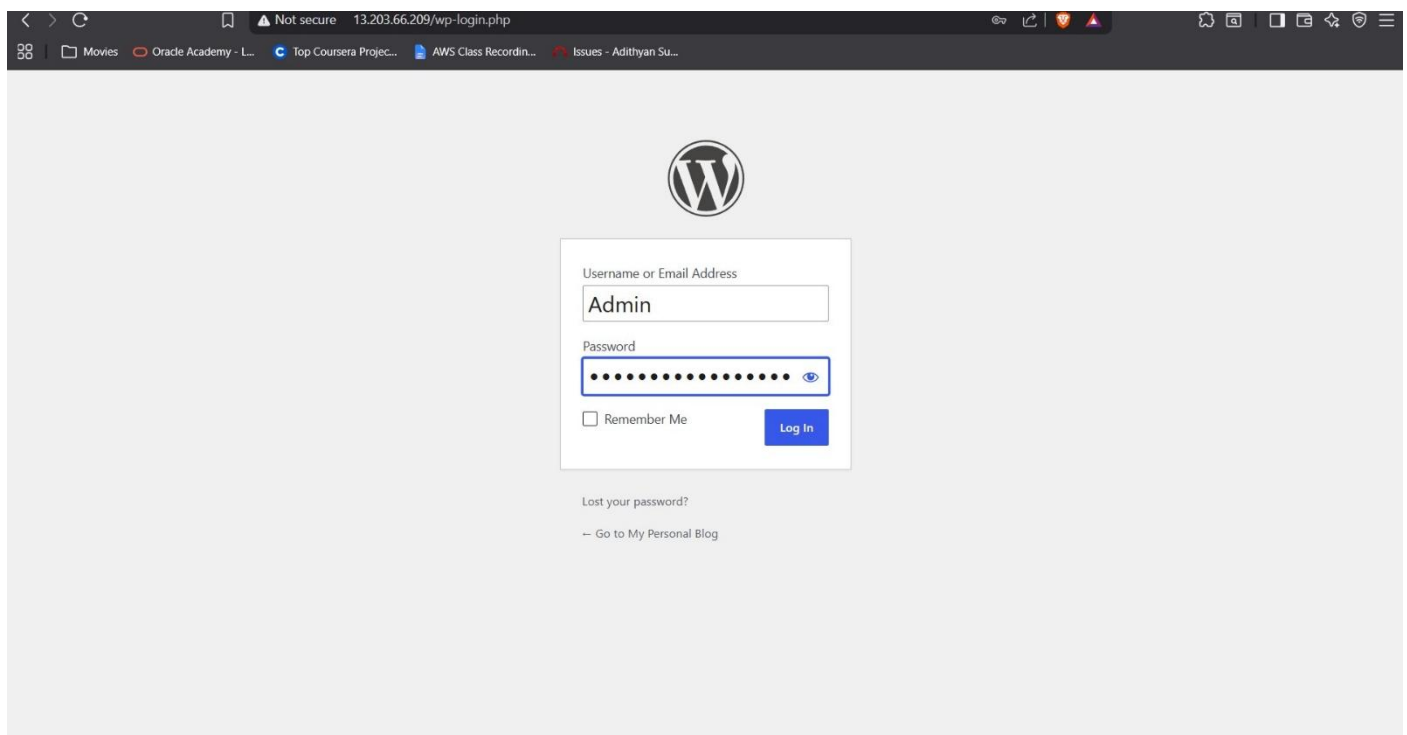
It is up to search engines to honor this request.

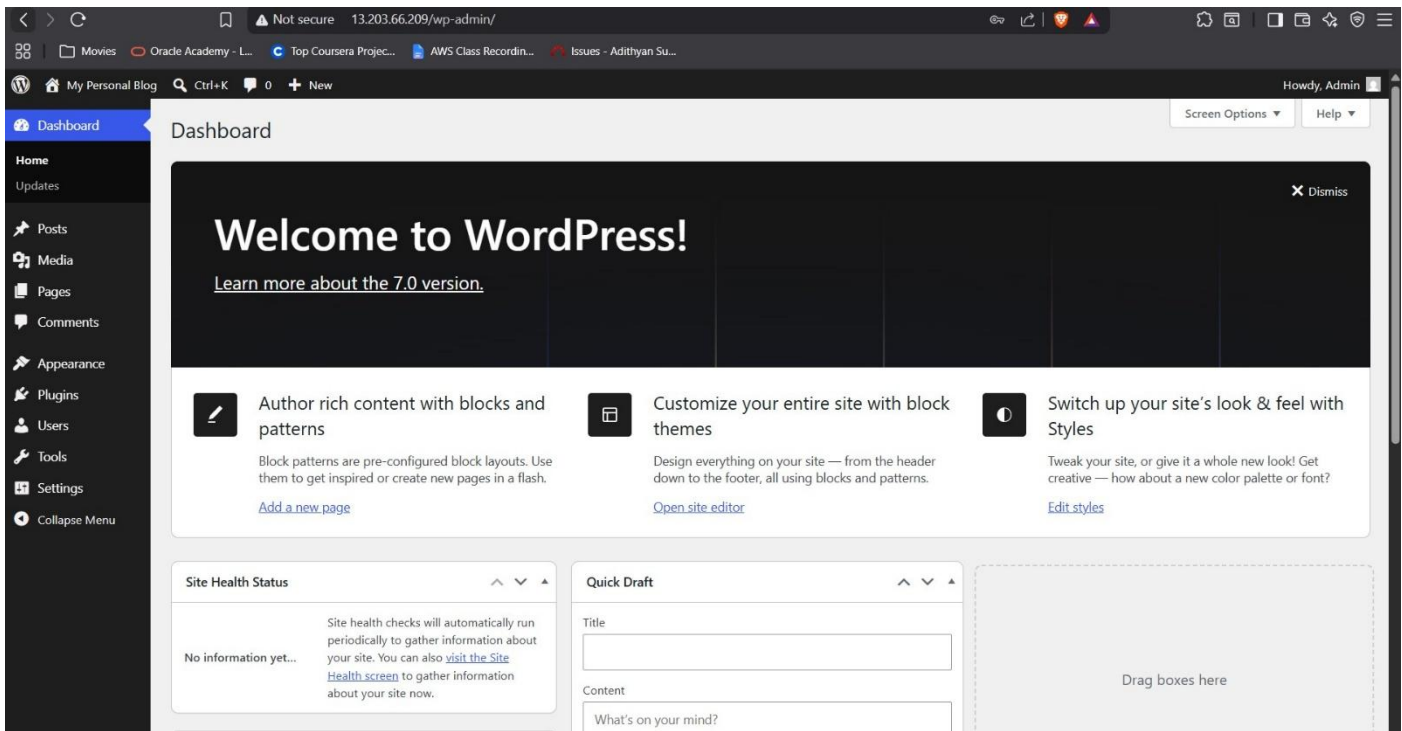
[Install WordPress](#)

Step 3: Verify Successful Installation

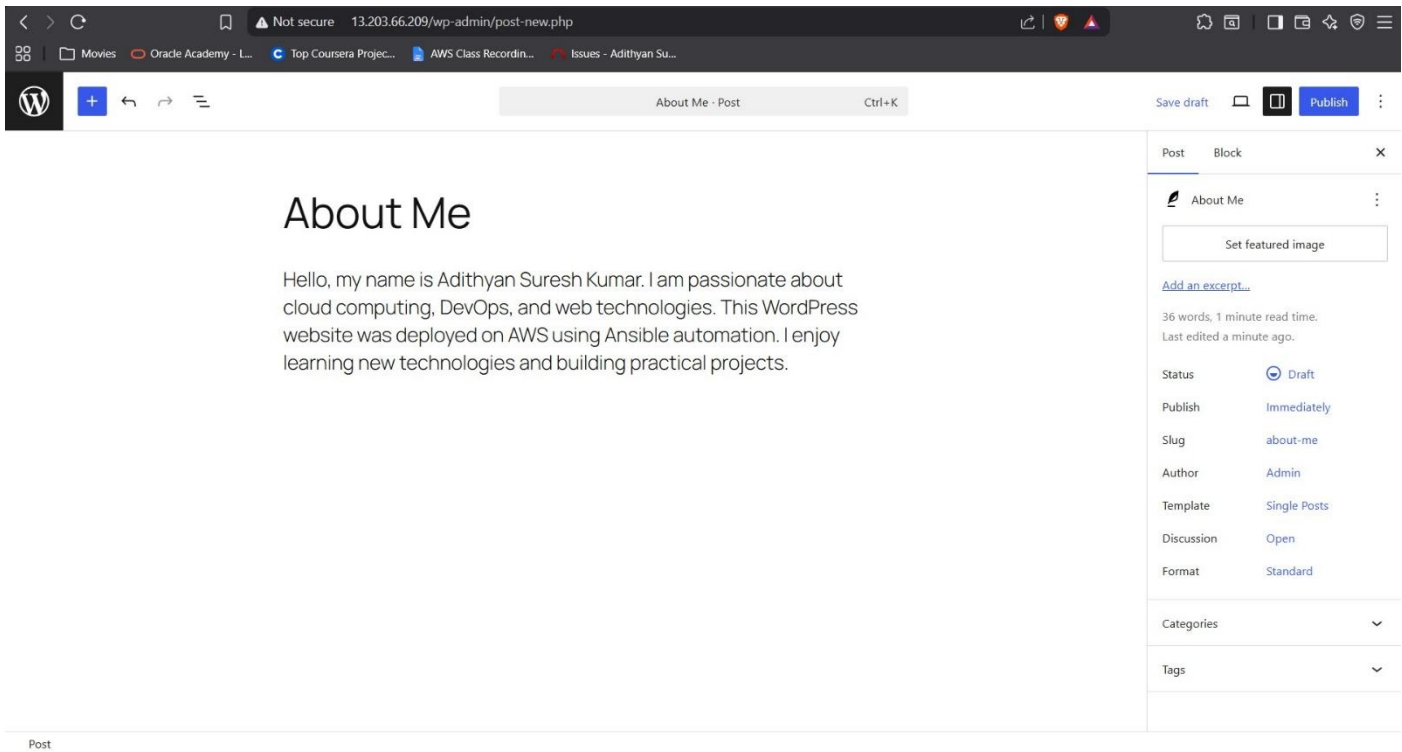


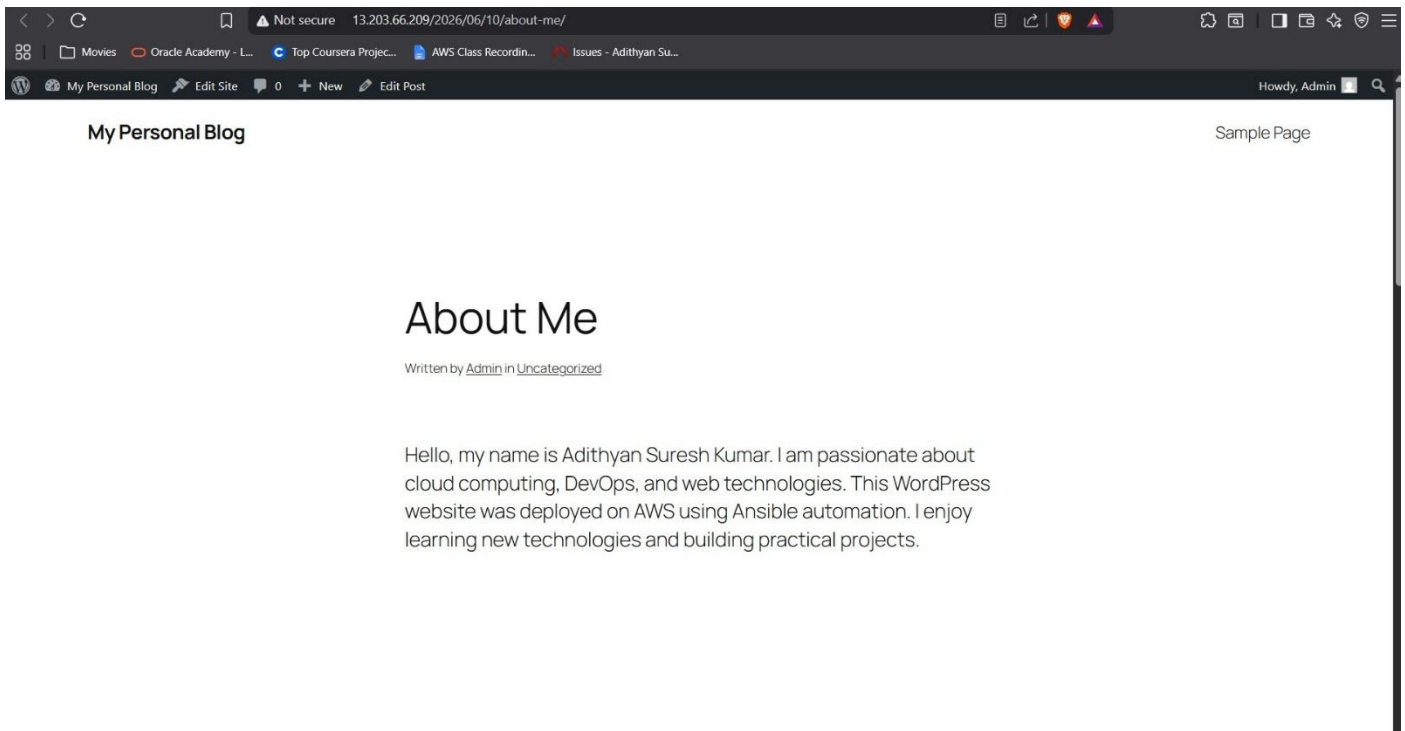
Step 4: Login to WordPress Dashboard





Step 5: Create Personal Blog Post





A personal blog post titled "About Me" was created and published successfully as required by the project instructions.

Task 7: User Access Configuration

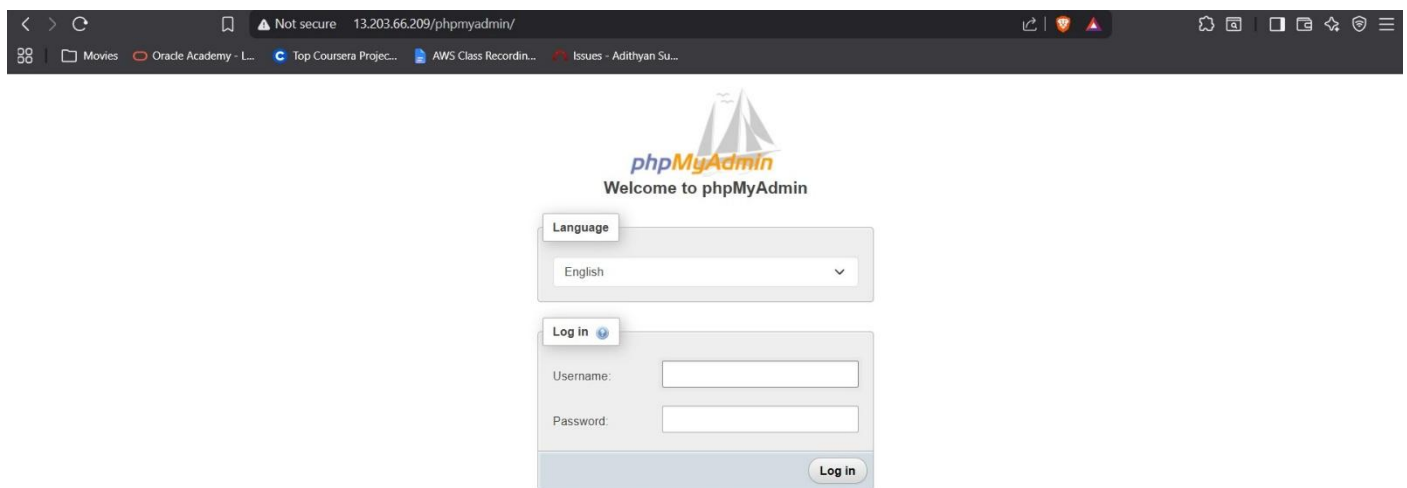
Step 1: Configure SFTP Access

```
[ec2-user@ip-172-31-12-92 public]$ sudo passwd websiteuser
Changing password for user websiteuser.
New password:
Retype new password:
passwd: all authentication tokens updated successfully.
[ec2-user@ip-172-31-12-92 public]$ sudo systemctl status sshd
● sshd.service - OpenSSH server daemon
   Loaded: loaded (/usr/lib/systemd/system/sshd.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-06-10 06:01:40 UTC; 1h 41min ago
     Docs: man:sshd(8)
           man:sshd_config(5)
   Main PID: 1627 (sshd)
    Tasks: 1 (limit: 1014)
   Memory: 7.3M
      CPU: 6.874s
   CGroup: /system.slice/sshd.service
           └─1627 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

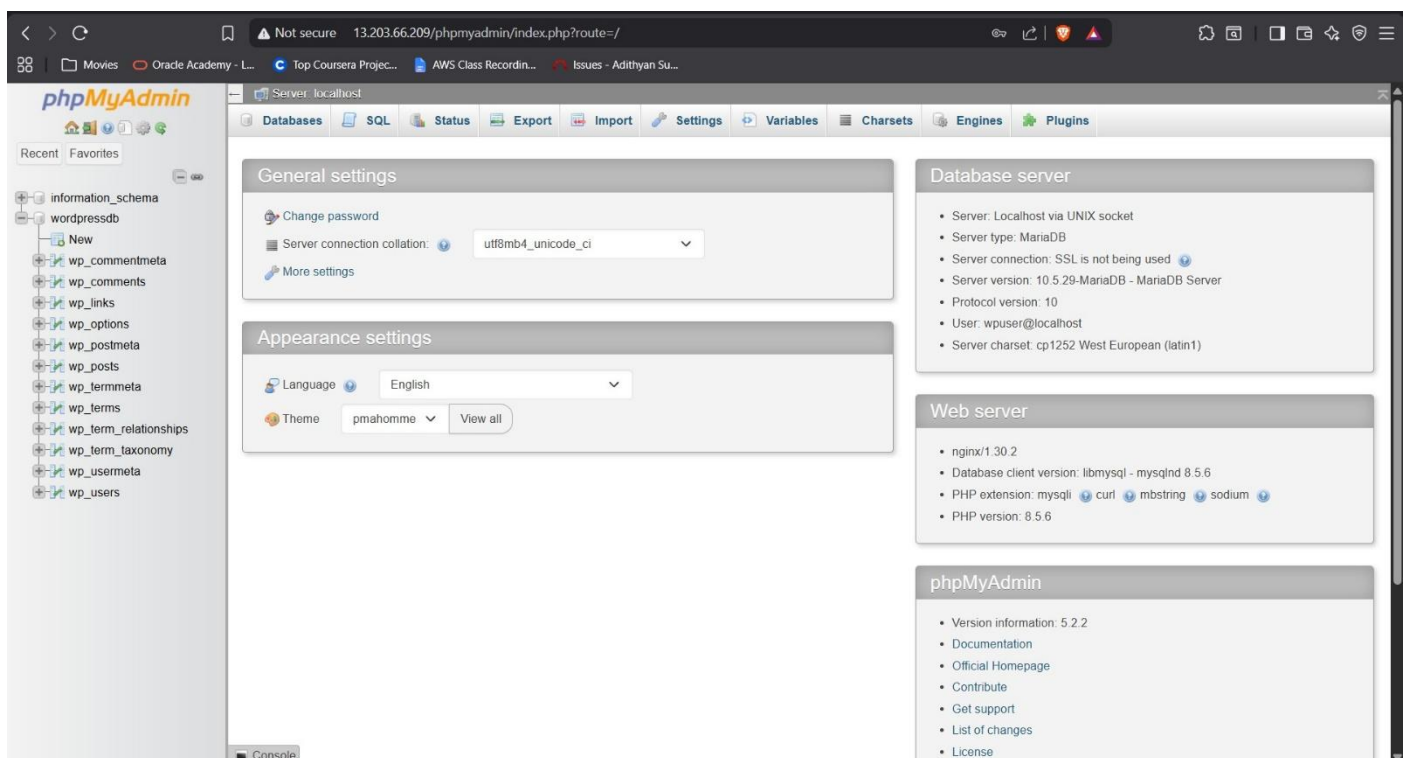
Jun 10 07:25:18 ip-172-31-12-92.ap-south-1.compute.internal sshd[34614]: Received disconnect from 103.149.26.43 port 51162:11: disconnected by user [preauth]
Jun 10 07:25:18 ip-172-31-12-92.ap-south-1.compute.internal sshd[34614]: Disconnected from authenticating user root 103.149.26.43 port 51162 [preauth]
Jun 10 07:31:22 ip-172-31-12-92.ap-south-1.compute.internal sshd[34957]: Received disconnect from 208.87.243.61 port 64942:11: disconnected by user [preauth]
Jun 10 07:31:22 ip-172-31-12-92.ap-south-1.compute.internal sshd[34957]: Disconnected from authenticating user root 208.87.243.61 port 64942 [preauth]
Jun 10 07:34:23 ip-172-31-12-92.ap-south-1.compute.internal sshd[35174]: Received disconnect from 188.44.20.30 port 59722:11: disconnected by user [preauth]
Jun 10 07:34:23 ip-172-31-12-92.ap-south-1.compute.internal sshd[35174]: Disconnected from authenticating user root 188.44.20.30 port 59722 [preauth]
Jun 10 07:38:02 ip-172-31-12-92.ap-south-1.compute.internal sshd[35411]: Received disconnect from 212.192.240.126 port 29302:11: disconnected by user [preauth]
Jun 10 07:38:02 ip-172-31-12-92.ap-south-1.compute.internal sshd[35411]: Disconnected from authenticating user root 212.192.240.126 port 29302 [preauth]
Jun 10 07:42:03 ip-172-31-12-92.ap-south-1.compute.internal sshd[35692]: Received disconnect from 191.101.33.110 port 53754:11: disconnected by user [preauth]
Jun 10 07:42:03 ip-172-31-12-92.ap-south-1.compute.internal sshd[35692]: Disconnected from authenticating user root 191.101.33.110 port 53754 [preauth]
[ec2-user@ip-172-31-12-92 public]$ getent passwd websiteuser
websiteuser:x:1001:1001::/home/websiteuser:/bin/bash
[ec2-user@ip-172-31-12-92 public]$
```

A dedicated user account (websiteuser) was created for website management. The OpenSSH service was verified to be active, enabling secure SFTP-based file transfers and remote file management through SSH authentication.

Step 2: Configure phpMyAdmin Access



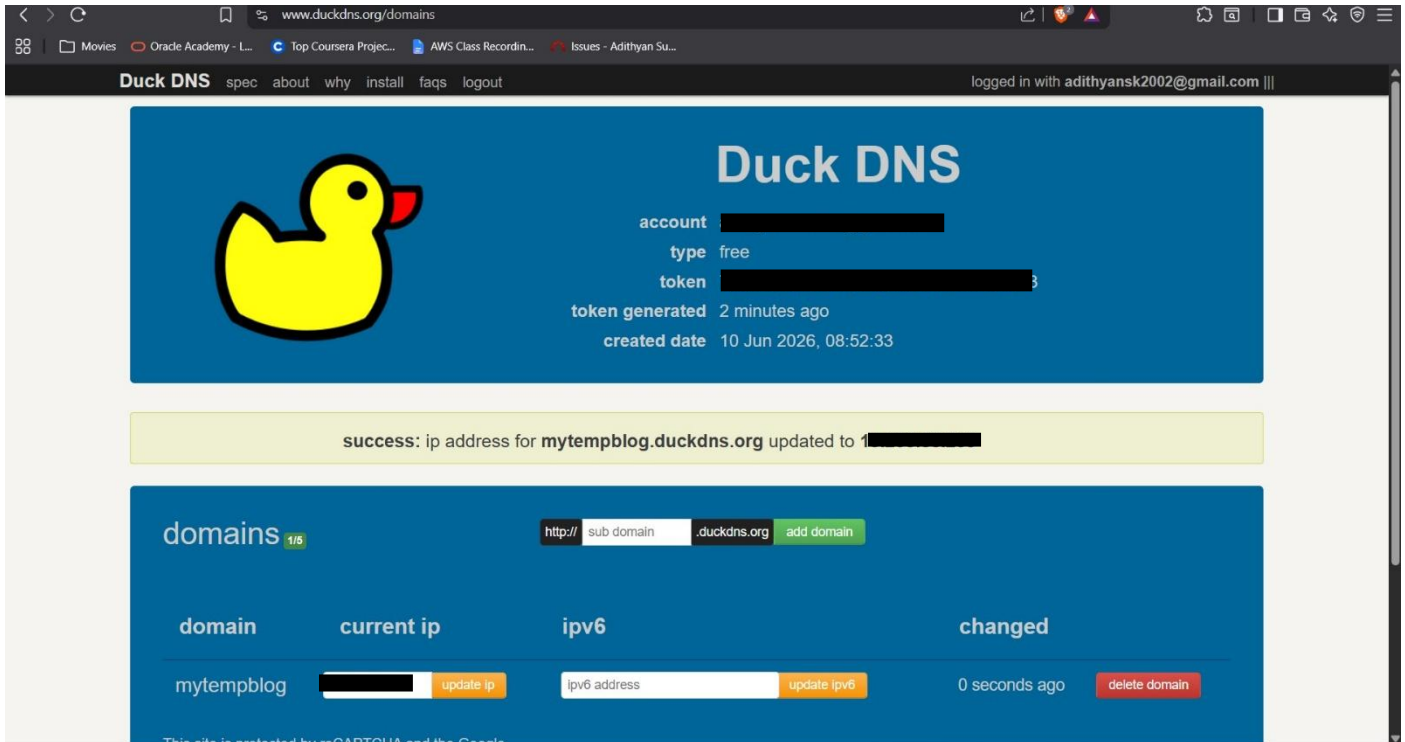
Step 3: Verify phpMyAdmin Functionality



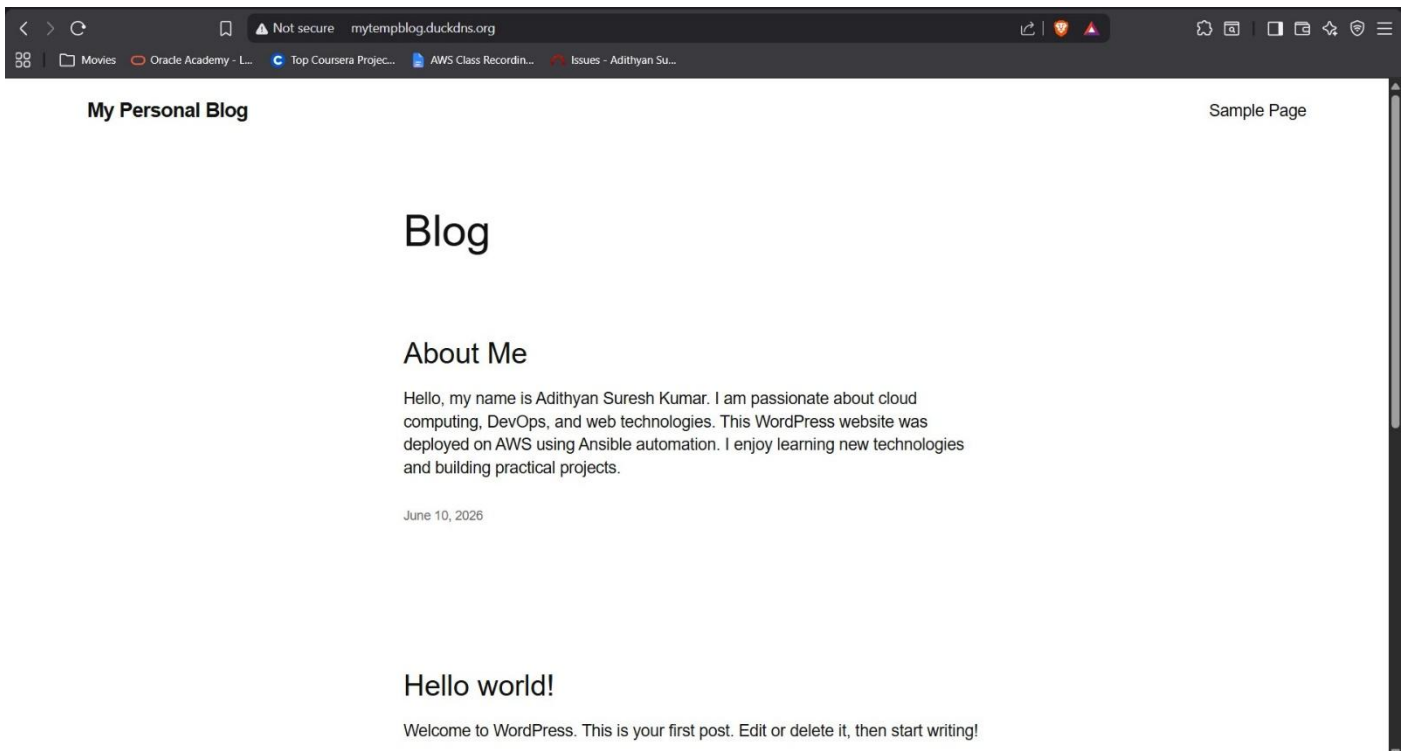
phpMyAdmin was successfully configured and integrated with MariaDB, allowing secure web-based database administration for the WordPress database.

Task 8: Domain and SSL Configuration

Step 1: Create DuckDNS Subdomain



Step 2: Verify Domain Resolution



The DuckDNS subdomain was successfully mapped to the EC2 public IP address, allowing users to access the website using a domain name instead of a direct IP address.

Step 3: Update Nginx Server Name Configuration

```
server {
    listen 80;
    server_name mytempblog.duckdns.org;
```

Step 4: Install Certbot

```
[ec2-user@ip-172-31-12-92 phpmyadmin]$ sudo dnf install certbot -y
Last metadata expiration check: 2:56:01 ago on Wed Jun 10 06:03:45 2026.
Dependencies resolved.

```

Package	Architecture	Version	Repository	Size
Installing: certbot	noarch	2.6.0-4.amzn2023.0.1	amazonlinux	49 k
Installing dependencies:				
fontawesome-fonts	noarch	1:4.7.0-11.amzn2023.0.2	amazonlinux	205 k
fonts-filesystem	noarch	1:2.0.5-12.amzn2023.0.2	amazonlinux	9.5 k
python3-acme	noarch	2.6.0-4.amzn2023.0.1	amazonlinux	161 k
python3-certbot	noarch	2.6.0-4.amzn2023.0.1	amazonlinux	677 k
python3-configargparse	noarch	1.7-1.amzn2023	amazonlinux	45 k
python3-josepy	noarch	1.13.0-6.amzn2023	amazonlinux	61 k
python3-parsedatetime	noarch	2.6-10.amzn2023	amazonlinux	80 k
python3-pyOpenSSL	noarch	21.0.0-1.amzn2023.0.2	amazonlinux	92 k
python3-pyrfc3339	noarch	1.1-16.amzn2023	amazonlinux	19 k
Installing weak dependencies:				
python-josepy-doc	noarch	1.13.0-6.amzn2023	amazonlinux	20 k

```
Transaction Summary
-----
Install 11 Packages

Total download size: 1.4 M
Installed size: 6.2 M
Downloading Packages:
(1/11): fonts-filesystem-2.0.5-12.amzn2023.0.2.noarch.rpm                238 kB/s | 9.5 kB  00:00
(2/11): certbot-2.6.0-4.amzn2023.0.1.noarch.rpm                      1.1 MB/s | 49 kB  00:00
(3/11): fontawesome-fonts-4.7.0-11.amzn2023.0.2.noarch.rpm           2.9 MB/s | 205 kB  00:00
(4/11): python-josepy-doc-1.13.0-6.amzn2023.0.2.noarch.rpm           437 kB/s | 20 kB  00:00
(5/11): python3-acme-2.6.0-4.amzn2023.0.1.noarch.rpm                  2.8 MB/s | 161 kB  00:00
```

A free SSL certificate was issued by Let's Encrypt using Certbot. Nginx was configured to serve the website over HTTPS and automatically redirect HTTP traffic to HTTPS.

Step 5: Generate SSL Certificate

```
[ec2-user@ip-172-31-12-92 phpmyadmin]$ dig mytempblog.duckdns.org +short
13.203.66.209
[ec2-user@ip-172-31-12-92 phpmyadmin]$ sudo systemctl stop nginx
[ec2-user@ip-172-31-12-92 phpmyadmin]$ sudo ss -tulpn | grep :80
[ec2-user@ip-172-31-12-92 phpmyadmin]$ sudo certbot certonly --standalone -d mytempblog.duckdns.org -v
Saving debug log to /var/log/letsencrypt/letsencrypt.log
Plugins selected: Authenticator standalone, Installer None
Requesting a certificate for mytempblog.duckdns.org
Performing the following challenges:
http-01 challenge for mytempblog.duckdns.org
Waiting for verification...
Cleaning up challenges

Successfully received certificate.
Certificate is saved at: /etc/letsencrypt/live/mytempblog.duckdns.org/fullchain.pem
Key is saved at: /etc/letsencrypt/live/mytempblog.duckdns.org/privkey.pem
This certificate expires on 2026-09-08.
These files will be updated when the certificate renews.
Certbot has set up a scheduled task to automatically renew this certificate in the background.

-----
If you like Certbot, please consider supporting our work by:
 * Donating to ISRG / Let's Encrypt: https://letsencrypt.org/donate
 * Donating to EFF: https://eff.org/donate-le
-----
```

Step 6: Configure Nginx for HTTPS

```
[ec2-user@ip-172-31-12-92 phpmyadmin]$ sudo cat /etc/nginx/conf.d/wordpress.conf
server {
    listen 80;
    server_name mytempblog.duckdns.org;

    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name mytempblog.duckdns.org;

    ssl_certificate /etc/letsencrypt/live/mytempblog.duckdns.org/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/mytempblog.duckdns.org/privkey.pem;

    root /home/websiteuser/myblog/public;
    index index.php index.html;

    location / {
        try_files $uri $uri/ /index.php?$args;
    }

    location ~ \.php$ {
        include fastcgi_params;
        fastcgi_pass unix:/run/php-fpm/www.sock;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
        include fastcgi.conf;
    }

    location ~ /\.ht {
        deny all;
    }
}
```

Step 7: Validate SSL Configuration

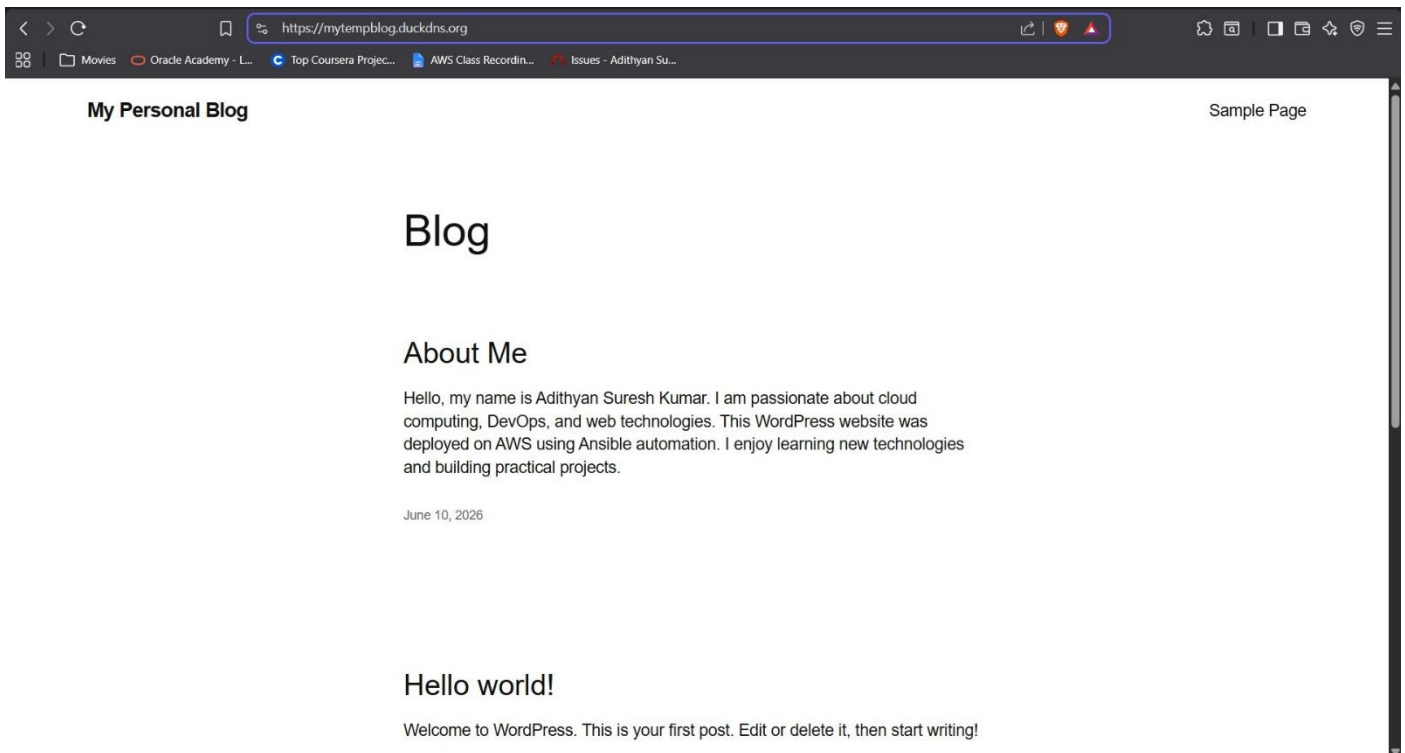
```
[ec2-user@ip-172-31-12-92 phpmyadmin]$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

Step 8: Restart Nginx Service

```
[ec2-user@ip-172-31-12-92 phpmyadmin]$ sudo systemctl start nginx
[ec2-user@ip-172-31-12-92 phpmyadmin]$ sudo systemctl restart nginx
[ec2-user@ip-172-31-12-92 phpmyadmin]$ sudo systemctl status nginx
• nginx.service - The nginx HTTP and reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: disabled)
   Drop-In: /usr/lib/systemd/system/nginx.service.d
            └─php-fpm.conf
   Active: active (running) since Wed 2026-06-10 09:16:28 UTC; 7s ago
   Process: 42655 ExecStartPre=/usr/bin/rm -f /run/nginx.pid (code=exited, status=0/SUCCESS)
   Process: 42658 ExecStartPre=/usr/sbin/nginx -t (code=exited, status=0/SUCCESS)
   Process: 42659 ExecStart=/usr/sbin/nginx (code=exited, status=0/SUCCESS)
   Main PID: 42660 (nginx)
   Tasks: 3 (limit: 1014)
   Memory: 3.8M
   CPU: 65ms
   CGroup: /system.slice/nginx.service
           └─42660 "nginx: master process /usr/sbin/nginx"
             └─42661 "nginx: worker process"
               └─42662 "nginx: worker process"

Jun 10 09:16:28 ip-172-31-12-92.ap-south-1.compute.internal systemd[1]: Starting nginx.service - The nginx HTTP and reverse proxy server...
Jun 10 09:16:28 ip-172-31-12-92.ap-south-1.compute.internal nginx[42658]: nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
Jun 10 09:16:28 ip-172-31-12-92.ap-south-1.compute.internal nginx[42658]: nginx: configuration file /etc/nginx/nginx.conf test is successful
Jun 10 09:16:28 ip-172-31-12-92.ap-south-1.compute.internal systemd[1]: Started nginx.service - The nginx HTTP and reverse proxy server.
```

Step 9: Verify HTTPS Website Access



The website was successfully accessed over HTTPS, confirming that SSL encryption and secure communication were functioning correctly.

Step 10: Verify SSL Certificate Details

```
[ec2-user@ip-172-31-12-92 phpmyadmin]$ sudo certbot certificates
Saving debug log to /var/log/letsencrypt/letsencrypt.log

-----
Found the following certs:
Certificate Name: mytempblog.duckdns.org
Serial Number: 68f77bcaa9c74623bd401aab5b4983c72f2
Key Type: ECDSA
Domains: mytempblog.duckdns.org
Expiry Date: 2026-09-08 08:12:53+00:00 (VALID: 89 days)
Certificate Path: /etc/letsencrypt/live/mytempblog.duckdns.org/fullchain.pem
Private Key Path: /etc/letsencrypt/live/mytempblog.duckdns.org/privkey.pem
-----
```

The SSL certificate was successfully issued and validated. The certificate is trusted by modern browsers and secures communication between users and the website using HTTPS encryption.

ANSIBLE AUTOMATION SUMMARY

The following tasks were automated using Ansible playbooks:

- Installation of Nginx
- Installation of PHP and required extensions
- Installation of MariaDB
- Website user creation
- Website directory creation
- WordPress deployment

- Database creation
- Database user creation
- Permission configuration

RESULTS AND DELIVERABLES

Requirement	Status
Nginx Installation	Completed
PHP Installation	Completed
MariaDB Configuration	Completed
WordPress Deployment	Completed
SFTP Configuration	Completed
phpMyAdmin Configuration	Completed
DuckDNS Domain Setup	Completed
SSL Certificate Installation	Completed
Personal Blog Publication	Completed

CONCLUSION

The project was successfully completed using Ansible automation on AWS EC2. The LEMP stack, WordPress, phpMyAdmin, SFTP access, DuckDNS domain, and Let's Encrypt SSL certificate were configured successfully. The website was secured with HTTPS and a personal blog post was published through the WordPress dashboard.